

Workshop Procedure: Unified Atomistic Workflows with ASE & DFT-FE

Platform: PARAM Siddhi AI (NVIDIA DGX SuperPOD) **Focus:** Universal, portable Python workflows for atomistic simulation.

This document outlines the step-by-step procedure for the hands-on session. We will execute four examples demonstrating a unified workflow across different physics engines (DFT-FE and MACE).

Note on Universality: While we are running these examples on the **PARAM Siddhi AI** supercomputer, the Python scripts (`ex*.py`) are **machine-agnostic**. The exact same code runs on laptops, workstations, or other supercomputers (Summit, Perlmutter) without modification. Only the batch submission script (`.slurm`) changes to match the local scheduler.

Table of Contents

- 0. Setup & Preparation
 - 1. Example 1: Ground State Calculation (DFT-FE)
 - 2. Example 2: Chained Optimization (DFT-FE)
 - 3. Example 3: MACE Calculator (ML Potential)
 - 4. Example 4: Molecular Dynamics (DFT-Driven)
 - 5. (Post-Session) Analyzing Existing Runs
 - Troubleshooting
-

0. Setup & Preparation

Goal: Prepare your local workspace with the clean example scripts.

1. **Open your terminal.**
2. **Copy the workshop materials** to your home directory: `bash cp -r /nlsasfs/home/gpucbh/vyakti50/dftfe_ase_new ~/ase_dftfe_workshop`
(Note: The source path `.../dftfe_ase_new` is the master copy prepared for you.)
3. **Navigate to the folder:** `bash cd ~/ase_dftfe_workshop ls`
You should see: `ex1.py`, `ex2.py`, `ex3.py`, `ex4.py` and their corresponding `.slurm` scripts.
4. **Important:** Open the scripts and **update the user placeholder** if necessary (though the scripts use `group_x/vyaktiXX` logic which should auto-detect or be safe).

- *Self-Check*: Ensure you are not overwriting someone else's files. The scripts are set to produce files like `ex1_group_x.log`.

1. Example 1: Ground State Calculation (DFT-FE)

Goal: Run a basic Self-Consistent Field (SCF) calculation on an Oxygen molecule (O_2).

1. **Inspect the Code**: `bash nano ex1.py`

- Observe the DFTFE calculator attachment.
- Note the simulation box size (40x42x38 Bohr) and `fermi_temp`.

Code Explanation:

- `atoms = Atoms(...)`: Creates the O_2 molecule object (Universal physics).
- `calc = DFTFE(...)`: Sets up the connection.
- `command=run_cmd`: **System Agnostic Interface**. detailed below:
 - `dftfe_bin`: Points to the executable. In production, we often use `os.environ` to make this dynamic.
 - `srun`: The MPI launcher. This can be `mpirun`, `jsrun`, or `mpiexec` depending on the machine. ASE handles them all through this single string.
- `atoms.get_potential_energy()`: The universal trigger.

2. **Run the Job**: `bash sbatch ex1.slurm`

3. **Monitor Progress**: `bash squeue -u $USER`

4. **Verify Results**:

- **Check SLURM Output** (for quick energy): `bash tail -n 20 ase_ex1_group_*.out` Look for: Total Free Energy: -31.76 Hartree.
- **Check Detailed Log** (for convergence): `bash grep "Total Free Energy" ex1_group_x.log` Verification: Energy should be approx -31.76 Hartree.

2. Example 2: Chained Optimization (DFT-FE)

Goal: Relax the bond length using a sequence of optimizers (FIRE $\rightarrow$ LBFGS).

1. **Inspect the Code**: `bash nano ex2.py`

- Note `compute_forces=True`.

- See the transition from FIRE optimizer to writing an intermediate file (.xyz).

Code Explanation:

- `compute_forces=True`: Critical for relaxation! We need forces, not just energy.
 - `opt1 = FIRE(atoms)`: Attaches the **Fast Inertial Relaxation Engine** (optimizer) to our atoms.
 - `opt1.run(fmax=0.05)`: Runs the optimization loop until the maximum force on any atom is below 0.05 eV/Angstrom.
2. **Run the Job:** `bash sbatch ex2.slurm` (*This may take a few minutes as it runs multiple DFT steps*)
 3. **Analyze Results:**
 - **Check Trajectory:** `bash grep "Step" ase_ex2_group_*.out`
 - **Visualize Intermediate Structure:** `bash ase gui ex2_stage1_relaxed.xyz`
 - **Measurement:** Select the two atoms. Distance should have relaxed from **1.27 Å**** to ≈ 1.22 Å**.

3. Example 3: MACE Calculator (ML Potential)

Goal: Run the *exact same* relaxation workflow, but swap the expensive DFT-FE calculator for a fast Machine Learning model (MACE).

1. **Inspect the Code:** `bash nano ex3.py`
 - Compare with `ex2.py`.
 - Notice `DFTFE(...)` is replaced by `MACECalculator(...)`.

Code Explanation:

- `atoms.calc = MACECalculator(...)`: The **only** line that changed. We swapped the physics engine.
 - `model_path`: Points to the pre-trained neural network weights.
 - Everything else (FIRE, `opt1.run`) remains exactly the same. ASE doesn't care *how* the forces are calculated, only *that* they are calculated.
2. **Run the Job:** `bash sbatch ex3.slurm`
 3. **Compare Performance:**
 - Check the runtime of `ex2` vs `ex3` using the SLURM output files.
 - **DFT-FE:** Minutes.
 - **MACE:** Seconds.

- **Accuracy Check:** The final bond length in **ex3** should match **ex2** very closely.

4. Example 4: Molecular Dynamics (DFT-Driven)

Goal: Run an NVT simulation at 300K driven by DFT-FE forces.

1. **Inspect the Code:** `bash nano ex4.py`

- Thermostat: **Langevin**.
- Timestep: 1.0 fs.
- Temperature: 300 K.

Code Explanation:

- `MaxwellBoltzmannDistribution(...)`: Randomizes initial atomic velocities to match 300K.
- `dyn = Langevin(...)`: Sets up the **Langevin thermostat** (simulating a heat bath).
- `dyn.run(steps=10)`: Moves the atoms in time. For each time step, it calls the calculator (DFT-FE) to get new forces.

2. **Run the Job:** `bash sbatch ex4.slurm`

3. **Analyze Trajectory:**

- **Visual Check:** `bash ase gui ex4_group_x.xyz` Press “Play” to see the thermal vibration of the bond.
- **Quantitative Check:** Open the file in OVITO (if available) or check the bond length oscillation in `ase gui`. It should vibrate around the equilibrium bond length (1.22 Å).

5. (Post-Session) Analyzing Existing Runs in ~/dftfe_ase

If you have already run the examples in the `~/dftfe_ase` directory during the session, follow these steps to review your generated data:

1. **Navigate to the Directory:** `bash cd ~/dftfe_ase ls -lt`
 - Use `ls -lt` to see the most recently created files at the top.
 - Identify your specific log files (e.g., `ex1_group_x.log`).
2. **Re-Check Example 1 (Energy):** `bash grep "Total Free Energy" ex1_group_*.log`
 - **Goal:** Confirm the final converged energy is approximately **-31.76 Hartree**.
3. **Re-Check Example 2 (Relaxation):**

- View the intermediate relaxed structure: `bash ase gui ex2_stage1_relaxed.xyz`
 - **Goal:** Measure the bond length. It should have relaxed from 1.27 Å to ~ 1.22 Å.
4. **Re-Check Example 3 (Performance Comparison):**
- Compare the runtime in the SLURM output files (.out): `bash grep "Wall time" ase_ex2_*.out grep "Wall time" ase_ex3_*.out`
 - **Goal:** Confirm that Example 3 (MACE) completed significantly faster than Example 2 (DFT-FE).
5. **Re-Check Example 4 (MD Trajectory):**
- Visualize the molecular dynamics trajectory: `bash ase gui ex4_group_*.xyz`
 - **Goal:** Observe the thermal vibration of the O-O bond.
-

Troubleshooting

- **MPI_ERRORS_ARE_FATAL:** This usually indicates an environment issue. Check that your submission script sources `setup_remote.sh` correctly.
 - **Job pending too long:** Check `squeue`. If the queue is full, ask the instructor.
 - **Missing Output:** Ensure you are looking in the directory where you submitted the job.
 - **“Socket connection failed”:** This can happen if the DFT-FE binary isn’t found. Verify paths in `ex1.py`.
-

Appendix: Running on Other Systems

The scripts are designed to be **Portable**. To run on a laptop or a different supercomputer (Summit, Perlmutter):

1. **Keep the Python Code:** The physics logic (Atoms, Optimizer, MD) is 100% identical.
 2. **Path Abstraction:** Variables like `dftfe_bin` are currently hardcoded for the workshop. In production, use `os.environ["DFTFE_BIN"]` to read locations dynamically.
 3. **Command Abstraction:** The `command` variable is just a shell string.
 - **Param Siddhi:** `srun -n 1 ...`
 - **Laptop:** `mpirun -n 1 ...`
 - **Summit:** `jsrun ...` By changing this one string (or auto-detecting it), the same script runs everywhere.
-